SBA301 – SLOT 28

CHAPTER 17 – HIBERNATE ARCHITECTURE, CONFIGURATION, MAPPING

PART A: ORM CONCEPT • SESSIONFACTORY • SESSION/LIFECYCLE • MAPPING • HQL

CORE: Tư duy trung tâm của Slot 28: Hibernate không chỉ là một thư viện CRUD. Nó là một ORM runtime quản lý mapping giữa object model và relational model, persistence context, entity lifecycle, transaction boundary và query translation. Sinh viên phải nhìn được chuỗi: Java object → Session/persistence context → mapping metadata → Hibernate SQL generation → JDBC → SQL Server, và hiểu SessionFactory/Session có lifecycle và trách nhiệm hoàn toàn khác nhau.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	28
Chủ đề	Chapter 17 Part A – Hibernate Architecture, Configuration, Mapping: ORM, SessionFactory, lifecycle, mapping, HQL
CLO chính	CLO5 – Apply Hibernate/JPA concepts for relational persistence and CRUD-oriented application architecture
Ngữ liệu xuyên suốt	Library Management – Student/Book – bridge sang Lab 07
Course baseline	JDK 21 • Maven • Hibernate ORM course track • Jakarta/JPA mapping concepts • SQL Server • IntelliJ
Thời lượng gợi ý	135 phút trên lớp + bài tập/AI verification ngoài giờ
Learning flow	Prepare → Understand → Model → Trace → Implement → Break → Diagnose → Verify → Transfer

NOTE: Scope lock: Slot 28 tập trung ORM concept, Hibernate architecture, SessionFactory/Session, entity lifecycle, configuration, mapping và HQL. Không đi sâu JavaFX UI của Lab 07; không biến buổi học thành Spring Data JPA review. Relationship mapping được giới thiệu đủ để hiểu mapping graph, nhưng CRUD repository/service đầy đủ và JavaFX integration được để cho các slot/lab kế tiếp.

CÁCH SỬ DỤNG GIÁO TRÌNH

Theo template Slot 22: Phần IV là knowledge base lý thuyết mở rộng; Phần VI là implementation workbook. Mỗi claim về lifecycle/query phải được kiểm bằng log SQL, debugger, Hibernate statistics hoặc database state. AI chỉ được dùng như hypothesis/mentor; không thay thế source code, SQL log và official documentation.

Pha	Sinh viên làm gì?	Evidence
Prepare	Ôn JPA/SQL/transaction; kiểm JDK/Maven/SQL Server.	Baseline checklist
Understand	Phân biệt ORM, JDBC, JPA, Hibernate, SessionFactory, Session.	Concept map
Model	Map Student/Book từ class ↔ table/PK/FK.	Mapping worksheet
Trace	Trace bootstrap và một persist/query transaction.	Execution trace
Implement	Tạo Hibernate standalone micro-project + HQL.	Source + SQL evidence
Break	Sai dialect/mapping/session/transaction/HQL.	Failure evidence
Diagnose	Đọc exception, SQL log, entity state, session scope.	Debug note
Verify	Đối chiếu object state ↔ rows ↔ generated SQL.	Test matrix
Transfer	So sánh Hibernate native API/JPA/Spring Data usage.	Architecture worksheet

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT

0.1. Kiến thức cần ôn lại

- Java OOP: class, object, constructor, encapsulation, equals/hashCode cơ bản.
- SQL Server: table, primary key, foreign key, SELECT/INSERT/UPDATE/DELETE, transaction.
- JDBC mental model: Connection → PreparedStatement → ResultSet; manual object mapping.
- JPA từ các slot trước: @Entity, @Id, @GeneratedValue, relationship annotation ở mức cơ bản.
- 3-layer/Repository architecture: persistence layer không chứa UI logic.
- Maven: dependency, build lifecycle, `pom.xml`.

0.2. Công cụ và baseline

Thành phần	Yêu cầu	Kiểm tra
JDK	21 course environment	java -version
Maven	3.6.3+ hoặc wrapper	mvn -v
IDE	IntelliJ IDEA	idea .
SQL Server	LibraryDB hoặc DB thực hành kết nối được	SSMS SELECT 1
Driver	Microsoft SQL Server JDBC	Maven resolve
Hibernate	Course-compatible version	dependency tree/build

Thành phần	Yêu cầu	Kiểm tra
Git	Checkpoint sạch	git status

0.3. Pre-class 3–2–1 Learning Note

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi cần kiểm chứng
Entity map table; transaction nhóm thao tác; JPA là persistence specification.	SessionFactory khác Session ra sao? Khi nào object detached?	Một `session.persist(student)` thực sự dẫn đến SQL INSERT ở thời điểm nào?

0.4. Pre-flight checklist

- SQL Server service chạy và login được.
- Tạo/kiểm `LibraryDB` hoặc database thực hành.
- IntelliJ nhận JDK/Maven đúng.
- Không lưu DB password thật vào Git.
- Có thể chạy một Maven `mvn test` hoặc project Java console baseline.
- Commit/tag gợi ý: `s28-before-hibernate`.

Baseline commands

```
java -version
mvn -v
git status

# Sau khi project đã được tạo:
mvn -q -DskipTests compile
mvn test
```

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1.1. Từ JDBC/JPA abstraction đến Hibernate runtime

Khái niệm	Câu hỏi chính	Slot 28 trả lời
ORM	Object và row khác nhau thế nào?	Mapping metadata chuyển đổi hai mô hình
Hibernate architecture	Ai giữ metadata/configuration?	SessionFactory + services
Session	Unit of Work nằm ở đâu?	Persistence context + first-level cache
Lifecycle	Object có trạng thái nào?	Transient / persistent / detached / removed
Mapping	Class/field/relationship map ra sao?	Annotations/XML metadata
HQL	Query object model thế nào?	Entity/property thay table/column

1.2. Mục tiêu học tập

1. Giải thích ORM và object-relational impedance mismatch bằng ví dụ Student/Book.
2. Phân biệt JPA/Jakarta Persistence với Hibernate ORM và Hibernate native API.
3. Mô tả Hibernate architecture: Configuration/bootstrapping, SessionFactory, Session, Transaction, ConnectionProvider, Query API, caches và dialect.
4. Giải thích vì sao SessionFactory thường là long-lived/thread-safe còn Session là short-lived/unit-of-work.
5. Trace entity lifecycle: transient → persistent/managed → detached → removed.
6. Cấu hình Hibernate cho SQL Server và giải thích ý nghĩa của URL, dialect, show_sql, format_sql, schema generation.
7. Map class/field/PK/column và nhận biết relationship annotations cơ bản.
8. Giải thích HQL khác SQL/JPQL ở mức cần thiết và viết SELECT/filter/order/paging/update query an toàn bằng parameters.
9. Nhận biết first-level cache, dirty checking, flush và transaction commit.
10. Chuyển kiến thức sang Lab 07 và phân biệt course Hibernate native style với current ecosystem/Jakarta Persistence direction.

1.3. Mental model của Slot 28

```
BOOTSTRAP
Configuration / hibernate.cfg.xml / properties / mapping metadata
        |
        v
SessionFactory  (long-lived, expensive, shared)
        |
        +---- openSession()
                    |
                    v
Session / Persistence Context  (short-lived Unit of Work)
        |
        +-- beginTransaction()
        |      |
        |      +-- persist / get / createQuery / remove
        |      +-- entity state tracking / first-level cache
        |      +-- dirty checking
        |
        +-- flush -> SQL generation -> JDBC -> SQL Server
        |
        +-- commit / rollback
        |
        +-- close -> managed entities become detached
```

1.4. Course-ready vs production-ready direction

Mức	Biểu hiện	Giới hạn
Beginner	One SessionFactory, open Session, CRUD/HQL works.	Có thể hard-code config, ít test.
Course-ready	Session/transaction scoped đúng, parameterized HQL, mapping rõ, SQL verified.	Đủ Slot 28/Lab bridge.
Production direction	Framework-managed EM/Session, pooling, migrations, observability, fetch plan, locking, batch/query tuning.	Không ép project lab nâng version giữa kỳ.

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

2.1. ORM Concept

ORM ánh xạ class/object/field/association sang table/row/column/FK. Nó giảm boilerplate JDBC nhưng không loại bỏ nhu cầu hiểu relational model và SQL.

2.2. Object-relational impedance mismatch

Khác biệt identity, association, inheritance, collection, nullability, transaction và query semantics giữa object model và relational model tạo ra các quyết định mapping.

2.3. JPA/Jakarta Persistence vs Hibernate

Jakarta Persistence là specification/API chuẩn; Hibernate là ORM implementation có cả native API. `EntityManagerFactory/EntityManager` tương đương gần nhất với `SessionFactory/Session`.

2.4. SessionFactory

Long-lived, expensive to build, thread-safe representation của mapping/runtime services; thường một instance cho một database/configuration.

2.5. Session

Short-lived, single-threaded unit-of-work/persistence context; giữ first-level cache và theo dõi entity changes.

2.6. Transaction

Biên atomicity; persistence operations phải được nhìn trong transaction boundary. Commit có thể trigger flush trước khi DB commit.

2.7. ConnectionProvider/Dialect

ConnectionProvider trừu tượng hóa JDBC connection supply; Dialect giúp Hibernate sinh SQL phù hợp DB.

2.8. Persistence Context

Không chỉ cache. Nó đảm bảo identity map cho entity managed và là nơi dirty checking xảy ra.

2.9. Entity Lifecycle

Transient chưa được quản lý; Persistent/Managed gắn context; Detached có identity nhưng không còn tracked; Removed chờ delete khi flush/commit.

2.10. Dirty Checking

Managed entity thay field có thể được update mà không gọi explicit update khi flush/commit.

2.11. Flush ≠ Commit

Flush đồng bộ persistence context với DB bằng SQL; commit kết thúc transaction. Flush có thể xảy ra trước query/commit tùy flush mode.

2.12. First-level cache

Session-level, bắt buộc; cùng ID trong cùng context thường resolve cùng managed instance.

2.13. Second-level cache awareness

SessionFactory-level optional cache. Không bật mặc định một cách mù quáng; cần provider/region/data consistency strategy.

2.14. Mapping annotations

@Entity, @Table, @Id, @GeneratedValue, @Column là nền tảng; relationship annotations mô tả associations.

2.15. Identity strategy

IDENTITY phù hợp SQL Server IDENTITY nhưng ảnh hưởng insert timing/batching; cần hiểu generation strategy theo DB.

2.16. HQL

Object-oriented query language dùng entity name/property names, không trực tiếp table/column names.

2.17. HQL parameters

Named parameters giảm lỗi nối chuỗi và injection risk; bind value thay vì concatenate.

2.18. Paging

`setFirstResult` và `setMaxResults`/API tương ứng giới hạn result window; phải có deterministic ORDER BY khi paging quan trọng.

2.19. Bulk HQL update/delete

Bulk operations bypass normal managed-entity dirty checking semantics và có thể làm persistence context stale; cần clear/refresh strategy awareness.

2.20. Session scope

Không dùng một Session singleton cho cả ứng dụng; Session là short-lived và không thread-safe.

2.21. Decision table – chọn abstraction nào?

Nhu cầu	Hibernate native	Jakarta Persistence	Spring Data JPA
Học runtime Hibernate trực tiếp	Rất phù hợp	Phù hợp nhưng abstraction hơn	Quá cao cho mục tiêu này
Portable standard API	Ít hơn	Tốt	Tốt qua JPA
SessionFactory/Session mental model	Trực tiếp	EMF/EM tương đương	Ẩn phần lớn
Repository boilerplate thấp	Tự viết	Tự viết/DAO	Rất thấp
HQL extensions	Tốt	JPQL chuẩn	JPQL/derived/native

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

3.1. Case study – Library Management

Object model	Relational model	Mapping concern
Student	STUDENTS	PK IDENTITY; email unique
Book	BOOKS	business attributes: isbn/title/author
Student.books	STUDENTS_BOOKS	many-to-many join table
Student.email	nvarchar/varchar column	length/nullability/unique
Student.marks	numeric column	Java/SQL type compatibility

3.2. Request/use-case flow trong desktop/service app

```
UI / Service
  -> Repository/DAO
      -> SessionFactory.openSession()
          -> Session.beginTransaction()
              -> HQL / get / persist / remove
              -> SQL generated
              -> JDBC
              -> SQL Server
          -> commit / rollback
      -> Session.close()
  <- DTO/entity result
```

3.3. Trade-off trong sản phẩm thật

Vấn đề	Naive design	Production direction
Session lifecycle	Mở một Session toàn app	Unit-of-work/request/service transaction scope
Schema	hibernate.hbm2ddl.auto=update production	Flyway/Liquibase migrations
Queries	`from Student` tải hết	filter/paging/fetch plan
Relationships	EAGER mọi association	LAZY + explicit fetch plan
Mapping	Entity = API DTO	Tách boundary khi cần
Performance	Chỉ nhìn Java code	SQL logs, query plan, N+1/batching metrics

3.4. Current ecosystem bridge

Course source trình bày Hibernate architecture truyền thống với `SessionFactory`/`Session` và `hibernate.cfg.xml`. Trong ecosystem hiện tại, Hibernate vẫn duy trì native API này đồng thời là một Jakarta Persistence provider. Hibernate ORM 7.4 là latest stable vào tháng 10/2026, tương thích Jakarta Persistence 3.2; Spring Boot 4.1 nằm trong compatibility matrix của series này. Course baseline không cần nâng version giữa kỳ; sinh viên chỉ cần hiểu migration direction.

IV. NỘI DUNG CHI TIẾT – KNOWLEDGE BASE NỀN TẢNG VÀ HỆ SINH THÁI HIBERNATE

READING GUIDE: Phần IV là lý thuyết/knowledge map. Không dùng như checklist thực hành. Mục tiêu là giúp sinh viên giải thích runtime behavior trước khi code.

4.1. Persistence problem và ORM

- Ứng dụng OO thao tác object graph; RDBMS lưu normalized rows và keys.
- Không thể 'xóa SQL khỏi tư duy': ORM sinh SQL nhưng database vẫn quyết định constraints, indexes, transactions và query plan.
- ORM tốt nhất khi domain model và relational model được thiết kế có chủ đích.

4.2. Impedance mismatch

- Identity: object reference vs primary key.
- Association: object reference/collection vs foreign key/join table.
- Inheritance: class hierarchy vs relational schema.
- Lifecycle: object allocation/GC vs persistent row lifetime.
- Granularity/type/null semantics cũng có khác biệt.

4.3. Hibernate as ORM + JPA provider

- Hibernate có native API `SessionFactory`/`Session` và implement Jakarta Persistence `EntityManagerFactory`/`EntityManager`.
- JPA portability không có nghĩa mọi Hibernate extension đều portable.

4.4. Bootstrap architecture

- Configuration metadata đến từ properties/XML/annotations.
- Bootstrap tạo ServiceRegistry/metamodel và cuối cùng SessionFactory.
- SessionFactory creation đắt; không tạo cho mỗi operation.

4.5. SessionFactory responsibilities

- Giữ mapping runtime metamodel, services, DB configuration, caches và factory để mở Session.
- Thread-safe/long-lived; thường một factory cho một database/configuration.
- Close SessionFactory khi application shutdown.

4.6. Session responsibilities

- Đại diện Unit of Work; wrap JDBC connection conceptually và tạo Transaction/Query.
- Giữ persistence context/first-level cache.
- Không thread-safe; nên short-lived.

4.7. Persistence context identity map

- Trong một Session, một DB identity thường tương ứng một managed entity instance.
- `session.get(Student.class, id)` lặp lại có thể dùng first-level cache thay vì query lại ngay.
- Identity map giúp consistency nhưng có memory cost nếu Session sống quá lâu.

4.8. Entity lifecycle state machine

- Transient: mới tạo, chưa attached.
- Persistent/managed: gắn Session, được dirty-check.
- Detached: từng có identity nhưng Session đóng/evict/detach.
- Removed: scheduled for deletion khi flush.

4.9. persist() và identifier

- `persist()` đưa transient entity vào managed state.
- Với IDENTITY, insert có thể xảy ra sớm để lấy generated key; timing khác generator khác.
- Không suy luận SQL timing chỉ từ tên method; quan sát logs.

4.10. get/find/query và managed state

- Entity được load trong Session thường managed.
- Changing managed fields không cần explicit update cho dirty checking path.
- Query result entities thuộc persistence context hiện tại.

4.11. detach/clear/close

- Detach một entity: Hibernate không còn track change.
- `clear()` detach toàn bộ context.
- `close()` kết thúc Session; các managed entities trở thành detached.

4.12. merge semantics

- Merge không 'reattach chính object detached' theo cách đơn giản; nó copy state vào managed instance và trả managed reference.
- Đây là điểm sinh viên phải phân biệt object reference cũ và return value của merge.

4.13. remove

- Remove đánh dấu managed entity để delete.
- Foreign key/cascade/orphan rules quyết định hành vi association; DB constraint vẫn là lớp cuối.

4.14. Flush lifecycle

- Flush chuyển pending entity state changes thành SQL.
- Auto flush có thể xảy ra trước commit hoặc query cần consistency.
- Flush thành công không đồng nghĩa transaction đã commit.

4.15. Transaction boundary

- Atomic unit of work; commit/rollback quản lý tính nhất quán.
- Không nên từng statement tự autocommit trong domain use case phức tạp.
- Exception handling phải rollback và close resource đúng.

4.16. Connection management

- Session không nên bị hiểu là raw Connection.
- Hibernate/connection provider/pool quản lý acquisition/release policy.
- Production thường dùng datasource/pool do framework/container quản lý.

4.17. Dialect

- Dialect mô tả SQL capabilities/syntax của database.
- Modern Hibernate có khả năng tự nhận diện nhiều dialect; course config vẫn dùng explicit dialect để sinh viên hiểu concept.
- Không chọn dialect sai chỉ vì application 'compile được'.

4.18. hibernate.cfg.xml

- File XML truyền thống cấu hình connection, dialect, schema generation và mapped classes.
- Phù hợp Lab 07/standalone teaching.
- Trong Spring Boot, configuration thường externalized bằng `application.properties/yaml`.

4.19. hbm2ddl/schema generation

- create/create-drop/update/validate có mục đích khác nhau.
- `update` tiện cho lab nhưng không phải schema migration governance production.
- Production nên dùng migration tool và `validate`/controlled schema policy.

4.20. Entity mapping fundamentals

- @Entity xác định persistent entity; @Table đổi table mapping; @Id xác định identity.
- @Column mô tả column name/nullability/length/unique nhưng DB migration mới là source-of-truth production tùy governance.

4.21. Access strategy

- Mapping có thể dựa field hoặc property/getter tùy vị trí @Id/annotations.
- Trộn access strategy thiếu chủ đích làm model khó hiểu.

4.22. Basic/Transient/Embedded/Enum/Lob/Version

- @Transient loại field khỏi persistence.
- @Embedded cho value object; @Enumerated cần lựa chọn STRING vs ORDINAL.
- @Version hỗ trợ optimistic locking; @Lob cho large object.

4.23. Relationship mapping

- @ManyToOne thường là phía giữ FK.
- @OneToMany thường dùng mappedBy để mô tả inverse side.

• @ManyToMany dùng join table; nhưng domain lớn có thể cần association entity thay vì raw many-to-many.

4.24. Owning side

• Owning side quyết định FK/join table update.
• `mappedBy` không tạo cột FK mới; nó chỉ ra property phía owning.
• Bidirectional association cần đồng bộ cả hai phía trong object model nếu application phụ thuộc object graph consistency.

4.25. Fetching

• LAZY trì hoãn load; EAGER yêu cầu dữ liệu có sẵn theo semantic nhưng SQL shape có thể khác.
• N+1 thường xuất hiện khi traverse collection/entity graph không có fetch plan.
• Không sửa N+1 bằng EAGER everywhere.

4.26. Cascading

• Cascade propagate entity operations, không phải database ON DELETE CASCADE mặc định.
• PERSIST/MERGE/REMOVE cần chọn theo aggregate ownership.
• Cascade REMOVE trên shared many-to-many có thể nguy hiểm.

4.27. First-level cache

• Session-scoped, luôn có.
• Repeated load cùng id trong context thường không query DB lần nữa.
• `clear()`/detach thay đổi behavior này.

4.28. Second-level cache

• SessionFactory-scoped, optional.
• Cần provider/cache region/concurrency strategy và data staleness policy.
• Không dùng như giải pháp mặc định cho slow query.

4.29. HQL mental model

• HQL query entity types và attributes, không trực tiếp table/column.
• `from Student s where s.email = :email` phải dùng Java property/entity names.
• HQL được compile/translate thành SQL theo mappings và dialect.

4.30. HQL vs JPQL

• JPQL là chuẩn Jakarta Persistence; HQL là superset/dialect giàu tính năng hơn trong Hibernate.
• Portable code ưu tiên JPQL subset khi cần; Hibernate-specific code có thể tận dụng HQL extensions.

4.31. Typed queries

• Modern API nên tạo typed query khi biết result type.
• Typed query giảm cast errors và làm contract rõ hơn.

4.32. Named parameters

• `:email`, `:id` giúp query rõ và chống việc nối dữ liệu user vào query string.
• Parameter binding không thay thế authorization/input validation nhưng giảm injection risk.

4.33. Paging/order

• Paging cần ORDER BY deterministic nếu muốn page ổn định.
• Offset paging đơn giản nhưng có cost khi offset lớn; keyset pagination là production extension.

4.34. Aggregation/projection

• HQL có select expressions, aggregate, joins và projections.
• Không phải query nào cũng nên trả full entity graph.

4.35. Bulk update/delete caveat

- Bulk HQL tác động trực tiếp DB và không synchronize managed entity state như normal dirty checking.
- Sau bulk operation cần hiểu clear/refresh để tránh stale persistence context.

4.36. Native SQL escape hatch

- Native SQL hữu ích khi cần DB-specific capabilities/tuning.
- Trade-off: portability/mapping complexity; không chuyển mọi query sang native chỉ vì HQL mới lạ.

4.37. Exception/resource handling

- Rollback transaction khi failure; close Session trong finally/try-with-resource pattern phù hợp.
- Không nuốt Hibernate/JDBC exception và tiếp tục dùng transaction đã thất bại.

4.38. Observability

- SQL logging giúp học nhưng production logging phải có policy.
- Hibernate statistics, slow query log, DB execution plan và application tracing giúp tìm N+1/latency.

4.39. Testing mapping

- Mapping test không chỉ kiểm compile; phải persist→reload→assert, relationship FK/join rows và constraints.
- Integration test với DB gần production giúp phát hiện dialect/schema mismatch.

4.40. Current ecosystem 2026

- Hibernate ORM 7.4 là latest stable series (7.4.13.Final ngày 2026-10-11) theo Hibernate official releases.
- 7.4 tương thích Jakarta Persistence 3.2, Java 17/21/25/26 và Spring Boot 4.1 theo compatibility page.
- Hibernate 6/7 HQL đã được tái thiết kế mạnh; official HQL guide là nguồn cập nhật ưu tiên.
- Course project không bắt buộc nâng version; hãy phân biệt Course Baseline và Current Production Radar.

V. KINH NGHIỆM THỰC TIỄN, TIPS VÀ DEBUGGING

Vấn đề	Triệu chứng	Nguyên nhân thường gặp	Chẩn đoán
No SessionFactory	Bootstrap fail	config/driver/mapping sai	startup exception + properties
Unknown entity	Query/persist fail	entity chưa mapped/scanned	mapping metadata
Table/column not found	SQLServerException	schema/name mismatch	generated SQL + DB schema
Session closed	LazyInitializationException / access fail	entity detached	session scope/fetch plan
No transaction	changes không persist hoặc exception	write outside tx	begin/commit/rollback
HQL syntax error	QuerySyntaxException/SemanticException	dùng table/column thay entity/property	HQL source
N+1	nhiều SELECT	lazy traversal loop	SQL count/log
Memory tăng	Session giữ quá nhiều managed entities	long-lived session/batch loop	flush+clear strategy
Stale entity	DB changed nhưng object cũ	first-level cache/bulk update	refresh/clear
Duplicate row graph	join fetch collection	cartesian multiplication	SQL/result shape

5.1. Debugging decision tree

```
Hibernate operation fails
|
+-- Bootstrap/startup fails?
|   +-- JDBC driver?
|   +-- URL/credentials?
|   +-- dialect?
|   +-- entity mapping?
|
+-- Query fails?
|   +-- HQL uses entity/property names?
|   +-- parameter name/type?
|   +-- mapping association path?
|
+-- No SQL emitted?
|   +-- entity managed?
|   +-- transaction active?
|   +-- flush/commit happened?
```

```
|
+-- Too many SQL statements?
|   +-- lazy loop / N+1?
|   +-- repeated sessions?
|   +-- missing fetch plan?
|
+-- Object changed but DB not changed?
    +-- detached entity?
    +-- transaction rollback?
    +-- merge return value ignored?
```

5.2. Tips

- Tạo SessionFactory một lần; mở/đóng Session theo unit-of-work.
- Write operations luôn nghĩ cùng transaction boundary.
- Bật SQL log cho học/debug, không phụ thuộc vào log như production observability duy nhất.
- HQL dùng entity/property names; SQL dùng table/column names.
- Không giữ Session sống qua UI lifetime chỉ để tránh LazyInitializationException.
- Đừng dùng `hbm2ddl.auto=update` như migration strategy production.
- Query performance phải được xác minh ở SQL/DB layer.
- Parameter binding thay vì nối chuỗi query.
- Đọc return value của `merge()`; không giả định detached instance tự trở thành managed.

VI. HƯỚNG DẪN STEP BY STEP THỰC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG

WORKBOOK RULE: Mỗi bước có mục tiêu, thao tác, code/config, lệnh chạy, verification và PASS/FAIL. Đây là workbook, khác hoàn toàn Phần IV.

6.1. Step 1 – Tạo database baseline

SQL Server

```
IF DB_ID(N'LibraryDB') IS NULL
    CREATE DATABASE LibraryDB;
GO
USE LibraryDB;
GO

-- Có thể để Hibernate tạo schema trong lab.
-- Verification:
SELECT DB_NAME() AS CurrentDatabase;
```

PASS: `CurrentDatabase = LibraryDB`.

6.2. Step 2 – Tạo Maven project

CMD

```
mkdir sba301-slot28-hibernate
cd sba301-slot28-hibernate
idea .

# Hoặc IntelliJ: File -> New -> Maven Project
```

Project structure gợi ý:

```
sba301-slot28-hibernate/
├─ pom.xml
└─ src/
   ├─ main/
   │  ├─ java/
```

```
│  │  └─ fu/edu/sba301/slot28/
│  │     ├─ entity/
│  │     ├─ util/
│  │     ├─ repository/
│  │     └─ app/
│  └─ resources/
│     └─ hibernate.cfg.xml
└─ test/java/...
```

6.3. Step 3 – Maven dependencies

pom.xml skeleton

```
<dependencies>
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>${hibernate.version}</version>
  </dependency>

  <dependency>
    <groupId>com.microsoft.sqlserver</groupId>
    <artifactId>mssql-jdbc</artifactId>
    <version>${mssql.version}</version>
  </dependency>

  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

VERSION: Dùng version phù hợp course/project. Không copy Hibernate 7.4 vào course nếu lab đang khóa version khác.

Verification commands

```
mvn -q -DskipTests compile
mvn dependency:tree
```

6.4. Step 4 – `hibernate.cfg.xml`

Course-style configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "https://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>

    <property name="hibernate.connection.driver_class">
      com.microsoft.sqlserver.jdbc.SQLServerDriver
    </property>

    <property name="hibernate.connection.url">
      jdbc:sqlserver://localhost:1433;
      databaseName=LibraryDB;
      encrypt=true;
      trustServerCertificate=true
    </property>

    <property name="hibernate.connection.username">sa</property>
    <property name="hibernate.connection.password">${DB_PASSWORD}</property>
```

```
    <property name="hibernate.dialect">
      org.hibernate.dialect.SQLServerDialect
    </property>

    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

    <mapping class="fu.edu.sba301.slot28.entity.Student"/>
    <mapping class="fu.edu.sba301.slot28.entity.Book"/>
  </session-factory>
</hibernate-configuration>
```

SECURITY: Nếu XML placeholder không được runtime resolve trong project standalone, dùng environment/system properties hoặc programmatic override. Không commit password thật.

6.5. Step 5 – Tạo Student entity

Student.java

```
@Entity
@Table(name = "STUDENTS")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 120)
    private String email;

    @Column(nullable = false)
    private String firstName;

    @Column(nullable = false)
    private String lastName;

    private double marks;

    public Student() {}

    public Student(String email, String firstName,
                   String lastName, double marks) {
        this.email = email;
        this.firstName = firstName;
        this.lastName = lastName;
        this.marks = marks;
    }

    // getters/setters
}
```

PASS: compile; entity có no-arg constructor và @Id.

6.6. Step 6 – Tạo Book entity

Book.java

```
@Entity
@Table(name = "BOOKS")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false, unique = true)
    private String isbn;

    @Column(nullable = false)
    private String title;

    private String author;

    public Book() {}

    // constructors/getters/setters
}
```

6.7. Step 7 – SessionFactory utility

HibernateUtil.java

```
public final class HibernateUtil {

    private static final SessionFactory SESSION_FACTORY =
        buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            Configuration configuration = new Configuration();
            configuration.configure("hibernate.cfg.xml");
            return configuration.buildSessionFactory();
        } catch (Exception ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return SESSION_FACTORY;
    }

    public static void shutdown() {
        SESSION_FACTORY.close();
    }

    private HibernateUtil() {}
}
```

Verification: đặt breakpoint vào `buildSessionFactory()`; application lifetime chỉ gọi một lần.

CHECKPOINT: PASS: SessionFactory build thành công. FAIL: đọc root cause JDBC/mapping/dialect.

6.8. Step 8 – Persist entity và trace lifecycle

Persist workflow

```
Student student =
    new Student("alice@fu.edu.vn", "Alice", "Nguyen", 8.5);

// transient
try (Session session =
         HibernateUtil.getSessionFactory().openSession()) {

    Transaction tx = session.beginTransaction();

    session.persist(student);   // persistent/managed
    System.out.println(student.getId());

    tx.commit();
```

```
}
// detached after session closes
```

Verification SQL:

```
SELECT TOP (20) *
FROM dbo.STUDENTS
ORDER BY id DESC;
```

PASS: row mới tồn tại; console SQL cho thấy INSERT.

6.9. Step 9 – Dirty checking experiment

```
try (Session session =
         HibernateUtil.getSessionFactory().openSession()) {

    Transaction tx = session.beginTransaction();

    Student s = session.get(Student.class, 1L);
    s.setMarks(9.0); // no explicit session.update()

    tx.commit();     // dirty checking -> UPDATE
}
```

PASS: SQL UPDATE xuất hiện khi flush/commit.

MENTAL MODEL: Nếu `s` detached trước khi sửa thì thay đổi không được tự tracking.

6.10. Step 10 – First-level cache experiment

```
try (Session session =
         HibernateUtil.getSessionFactory().openSession()) {

    Student a = session.get(Student.class, 1L);
    Student b = session.get(Student.class, 1L);

    System.out.println(a == b);
}
```

Expected: cùng persistence context thường `true`, và SQL SELECT chỉ cần xuất hiện một lần cho cùng identity.

6.11. Step 11 – Detached experiment

```
Student detached;
try (Session session =
         HibernateUtil.getSessionFactory().openSession()) {
    detached = session.get(Student.class, 1L);
}

detached.setMarks(7.0); // detached change
// Không có transaction/session tracking -> DB chưa đổi
```

Verification: SELECT DB row, chứng minh Java object thay nhưng row không đổi.

6.12. Step 12 – Merge detached entity

```
try (Session session =
         HibernateUtil.getSessionFactory().openSession()) {

    Transaction tx = session.beginTransaction();

    Student managed = session.merge(detached);
    managed.setMarks(8.8);

    tx.commit();
}
```

IMPORTANT: Sử dụng object return từ `merge()` như managed reference. Detached object gốc không nên được giả định là reattached.

6.13. Step 13 – HQL SELECT cơ bản

```
try (Session session =
         HibernateUtil.getSessionFactory().openSession()) {

    List<Student> students =
        session.createQuery(
            "from Student s order by s.lastName",
            Student.class
        ).getResultList();

    students.forEach(System.out::println);
}
```

PASS: HQL dùng `Student` và `lastName` (entity/property), không dùng `STUDENTS` và column DB.

6.14. Step 14 – HQL WHERE + named parameter

```
List<Student> result =
    session.createQuery(
        "from Student s where s.email = :email",
        Student.class
    )
    .setParameter("email", "alice@fu.edu.vn")
    .getResultList();
```

SECURITY: Không nối input user vào HQL string.

6.15. Step 15 – HQL paging

```
List<Student> page =
    session.createQuery(
        "from Student s order by s.id",
        Student.class
    )
    .setFirstResult(0)
    .setMaxResults(10)
    .getResultList();
```

Verification: tối đa 10 rows; ORDER BY tạo pagination deterministic.

6.16. Step 16 – HQL projection

```
List<String> emails =
    session.createQuery(
        "select s.email from Student s order by s.email",
        String.class
    )
    .getResultList();
```

Mục tiêu: không phải query nào cũng cần load full entity.

6.17. Step 17 – HQL bulk UPDATE

Modern Hibernate-style example

```
try (Session session =
         HibernateUtil.getSessionFactory().openSession()) {

    Transaction tx = session.beginTransaction();

    int count = session.createMutationQuery(
        "update Student s set s.marks = :m where s.id = :id"
    )
```

```
    .setParameter("m", 9.5)
    .setParameter("id", 1L)
    .executeUpdate();

    tx.commit();
    System.out.println("updated=" + count);
}
```

VERSION: API `createMutationQuery()` là current Hibernate style. Nếu course version cũ dùng `createQuery(...).executeUpdate()`, giữ course API nhưng hiểu cùng bulk-mutation semantics.

6.18. Step 18 – Mapping Many-to-Many bridge

```
@ManyToMany
@JoinTable(
    name = "STUDENTS_BOOKS",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "book_id")
)
private Set<Book> books = new HashSet<>();
```

Verification: Hibernate tạo/dùng join table; kiểm `STUDENTS_BOOKS` sau persist association.

6.19. Step 19 – Transaction rollback experiment

```
try (Session session =
         HibernateUtil.getSessionFactory().openSession()) {

    Transaction tx = session.beginTransaction();

    try {
        Student s = new Student(
            "duplicate@fu.edu.vn", "A", "B", 5.0);
        session.persist(s);

        // deliberately cause another failure here

        tx.commit();
    } catch (Exception ex) {
        tx.rollback();
        throw ex;
    }
}
```

PASS: transaction atomicity được kiểm bằng DB state.

6.20. Step 20 – Break-It: sai HQL entity/table name

```
// Sai về HQL nếu entity name là Student:
session.createQuery("from STUDENTS", Student.class);
```

Expected: semantic/query exception. Fix thành `from Student`.

DIAGNOSE: HQL query object model; SQL query relational model.

6.21. Step 21 – Break-It: Session quá dài

Tạo loop load hàng nghìn entity trong cùng Session và quan sát memory/Session first-level cache. Sau đó refactor batch với `flush()` + `clear()` ở checkpoint phù hợp. Mục tiêu là hiểu Session lifetime, không benchmark tuyệt đối.

6.22. Step 22 – SQL verification matrix

Test	Operation	Expected SQL/DB
T01	persist Student	INSERT + row
T02	get same ID twice	SELECT lần đầu; first-level cache sau đó
T03	managed field change	UPDATE at flush/commit
T04	detached field change	No UPDATE
T05	merge detached	SELECT/UPDATE tùy state + DB changed

Test	Operation	Expected SQL/DB
T06	HQL parameter query	SELECT with bound parameter
T07	paging	DB-specific limit/offset translation
T08	many-to-many attach	join table row
T09	rollback	No committed partial change

6.23. Step 23 – Integration Gate G1–G13

Gate	Evidence
G1 Build	Maven compile/test pass
G2 Bootstrap	SessionFactory built once
G3 DB	LibraryDB connection verified
G4 Mapping	STUDENTS/BOOKS schema correct
G5 Lifecycle	transient/persistent/detached evidence
G6 Transaction	commit + rollback evidence
G7 Dirty checking	managed change → UPDATE
G8 Cache	same ID/session evidence
G9 HQL	select/filter/parameter
G10 Paging	ordered page
G11 Relationship	join table evidence
G12 Break/Fix	mapping/HQL/session failure diagnosed
G13 Explain	SessionFactory ≠ Session; flush ≠ commit

6.24. Bài tập tổng hợp hệ thống

1. Hoàn thiện Student và Book mapping.
2. Bootstrap một SessionFactory dùng chung.
3. Viết Repository/DAO methods sử dụng short-lived Session.
4. Persist ít nhất 5 Students và 5 Books.
5. Tạo association Student–Book và kiểm join table.
6. Viết HQL list/filter/order/page.
7. Thực hiện dirty-checking experiment.
8. Thực hiện detached + merge experiment.
9. Thực hiện rollback experiment.
10. Tạo README mô tả lifecycle, config, HQL và known limitations.
11. Viết Current Production Note: course native Hibernate → Jakarta Persistence/framework-managed transaction.

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ Slot 28
ORM	Object-Relational Mapping	Java object ↔ row
Hibernate ORM	Java ORM framework/JPA provider	hibernate-core
JPA/Jakarta Persistence	Persistence specification/API	EntityManager
SessionFactory	Long-lived Hibernate factory/runtime	openSession()
Session	Short-lived persistence context/unit of work	session.get()
Persistence Context	Managed entity context/identity map	first-level cache
Transaction	Atomic unit of work	commit/rollback
Transient	New object not managed	new Student()
Persistent/Managed	Entity attached to context	after persist/get
Detached	Entity no longer attached	after session.close()
Removed	Entity scheduled for delete	session.remove()
Dirty Checking	Detect managed changes	UPDATE at flush
Flush	Synchronize context with DB	SQL emission
First-level Cache	Session-scoped cache	same id/session
Second-level Cache	SessionFactory-scoped optional cache	shared regions
Dialect	DB SQL capability adapter	SQLServerDialect
Configuration	Hibernate bootstrap metadata	hibernate.cfg.xml
Mapping	Object↔relational metadata	@Table/@Column
@Entity	Persistent entity marker	Student
@Id	Entity identifier	id
@GeneratedValue	Identifier generation	IDENTITY
@ManyToOne	Many entities reference one	Order→Customer
@OneToMany	Collection side relationship	Category→Products
@ManyToMany	Many-to-many association	Student↔Book
Owning Side	Side responsible for relation update	@JoinTable side
HQL	Hibernate Query Language	from Student
JPQL	Jakarta Persistence query language	portable subset/concept
Query	Executable query abstraction	createQuery
Named Parameter	Bound query variable	:email
Projection	Select subset/scalar/DTO	select s.email
Pagination	Limit result window	setFirstResult/setMaxResults

Thuật ngữ	Định nghĩa	Ví dụ Slot 28
Bulk Mutation	Direct HQL update/delete	createMutationQuery
Lazy Loading	Deferred association load	collection proxy
N+1	One query + many follow-up queries	loop association
Optimistic Lock	Version-based concurrency control	@Version

VIII. RESEARCH BASE VÀ AI-BASED LEARNING – 4 NHÓM TRẢI NGHIỆM CUỐI BUỔI

8.1. Quy tắc dùng AI

- AI phải đọc source/config thực tế; không được bịa class/method.
- Execution/lifecycle claim phải kiểm bằng breakpoint, SQL log hoặc DB state.
- Version/current recommendation phải kiểm official Hibernate/Jakarta docs.
- Không đưa DB password/credential thật vào prompt.
- Phân biệt Course Baseline với Current Production Direction.

8.2. NHÓM I – Workflow / execution rule

PROMPT I.1: Trace bootstrap từ `Configuration.configure()` đến SessionFactory; liệt kê metadata/services hình thành.

PROMPT I.2: Trace `openSession()` → beginTransaction → persist → flush → commit → close; chỉ rõ entity state từng bước.

PROMPT I.3: Trace một `session.get(Student.class, id)` và giải thích first-level cache bằng hai lần gọi cùng id.

PROMPT I.4: Trace dirty checking: field change lúc managed nhưng không explicit update; xác định thời điểm SQL phát ra.

PROMPT I.5: Trace detached → merge và chỉ rõ object nào managed sau merge.

PROMPT I.6: Trace HQL từ string → semantic model → SQL generation → JDBC → ResultSet → entities.

PROMPT I.7: Trace many-to-many persist đến INSERT join table.

PROMPT I.8: Trace rollback và chứng minh DB state không bị partial commit.

PROMPT I.9: Trace query paging và tìm DB-specific SQL do dialect sinh.

PROMPT I.10: Vẽ dependency/runtime graph: SessionFactory → Session → Transaction/Query → JDBC/DB.

8.3. NHÓM II – 10 prompt kỹ thuật nâng cao

PROMPT II.1: Phân tích `persist`, `merge`, `update` (legacy/native), `saveOrUpdate` nếu version có: entity-state semantics và migration direction.

PROMPT II.2: Giải thích flush modes AUTO/COMMIT/MANUAL và thiết kế thí nghiệm quan sát SQL timing.

PROMPT II.3: Phân tích first-level cache identity map và memory growth; đề xuất batch flush/clear strategy.

PROMPT II.4: Phân tích N+1 trên Student–Book và so sánh join fetch, entity graph, batch fetching.

PROMPT II.5: Phân tích owning side/mappedBy/cascade/orphanRemoval bằng 5 mutation scenarios.

PROMPT II.6: So sánh IDENTITY, SEQUENCE, TABLE và UUID generation về batching/performance/portability, tập trung SQL Server.

PROMPT II.7: Phân tích optimistic locking @Version: lost update scenario, SQL WHERE version và exception flow.

PROMPT II.8: Phân tích HQL bulk update làm persistence context stale; thiết kế fix bằng clear/refresh.

PROMPT II.9: Phân tích equals/hashCode cho Hibernate entities và các rủi ro khi dùng generated id + Set.

PROMPT II.10: Thiết kế benchmark/query-plan experiment so sánh HQL entity load, projection và native SQL.

8.4. NHÓM III – 10 prompt production engineering

PROMPT III.1: Review architecture DAO mở Session thủ công; đề xuất transaction/session boundary cho desktop app, web app và Spring app.

PROMPT III.2: Thiết kế migration từ hbm2ddl.auto=update sang Flyway/Liquibase + validate; nêu rollback/deployment process.

PROMPT III.3: Thiết kế connection pool policy HikariCP: pool size, timeout, leak detection và đo evidence thay vì đoán.

PROMPT III.4: Thiết kế Hibernate observability: SQL logging, statistics, Micrometer/JFR, DB slow query và correlation.

PROMPT III.5: Review fetch strategy cho e-commerce Order/OrderItems/Product; tránh N+1 và cartesian explosion.

PROMPT III.6: Thiết kế batch insert/update 100k rows: JDBC batching, ID strategy, flush/clear cadence và transaction size.

PROMPT III.7: Thiết kế caching policy: first-level, second-level, query cache; dữ liệu nào cache/không cache và invalidation risks.

PROMPT III.8: Thiết kế concurrency policy: optimistic vs pessimistic locking cho inventory/payment examples.

PROMPT III.9: Thiết kế repository/query governance: HQL/JPQL/native/Criteria; code-review rules và performance evidence.

PROMPT III.10: Threat-model persistence layer: parameterized HQL, native SQL, mass assignment, sensitive fields, DB credentials và audit.

8.5. NHÓM IV – 10 prompt current ecosystem / community trends

PROMPT IV.1: Kiểm tra official Hibernate ORM latest stable hiện tại, Java/Jakarta Persistence/Spring Boot compatibility và ghi ngày truy cập.

PROMPT IV.2: Đọc Hibernate 7.4 User Guide architecture; so sánh SessionFactory/Session với cách course slide mô tả. Chỉ ra điểm nào còn đúng, điểm nào cần cập nhật.

PROMPT IV.3: Đọc official HQL guide cho Hibernate 7.4 và liệt kê 10 capability HQL hiện đại vượt ví dụ course cơ bản.

PROMPT IV.4: Khảo sát Hibernate Data Repositories/Jakarta Data 1.0 và so sánh mental model với Spring Data JPA repositories.

PROMPT IV.5: Khảo sát Hibernate Processor compile-time tooling/type-safe query support và giá trị cho refactoring.

PROMPT IV.6: Kiểm tra status Hibernate ORM 8.0 development và Jakarta Persistence 4.0; nêu WATCH LATER, không khuyến nghị production nếu chưa stable.

PROMPT IV.7: Khảo sát Hibernate Reactive và giải thích vì sao Session blocking model không thể chỉ 'đổi async' mà xong.

PROMPT IV.8: Khảo sát current dialect support cho SQL Server trong Hibernate official dialect report; nêu version compatibility.

PROMPT IV.9: Khảo sát `@SoftDelete`, `@TenantId`, filters hoặc modern Hibernate features liên quan SaaS multi-tenant/data lifecycle; phân loại SHOULD EXPLORE.

PROMPT IV.10: Tạo '2026 Hibernate Persistence Radar': ORM 7.4, Jakarta Persistence 3.2, HQL, Data Repositories, migration tools, caching, observability, reactive, type-safe queries.

8.6. AI Verification Worksheet

Claim	Nguồn kiểm chứng	Experiment	Kết luận SV
SessionFactory long-lived	official Hibernate docs	count bootstrap calls	
Dirty checking timing	source + SQL log	managed field change	
HQL translation	HQL guide + SQL log	parameter query	
N+1 suspicion	SQL count/DB	association loop	
Current Hibernate version	hibernate.org official	dependency sample	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

□ 1. Tôi giải thích được ORM và impedance mismatch.

□ 2. Tôi phân biệt JPA/Jakarta Persistence với Hibernate.

□ 3. Tôi phân biệt SessionFactory và Session theo lifecycle/thread safety/cost.

□ 4. Tôi vẽ được entity state transitions.

□ 5. Tôi giải thích dirty checking và flush khác commit.

□ 6. Tôi biết first-level cache thuộc Session.

□ 7. Tôi cấu hình được SQL Server/Hibernate baseline.

□ 8. Tôi map được Entity/Id/Column và relationship cơ bản.

□ 9. Tôi hiểu owning side và join table.

□ 10. Tôi viết HQL bằng entity/property names.

□ 11. Tôi dùng named parameters thay nối chuỗi.

□ 12. Tôi thực hiện paging có ORDER BY.

□ 13. Tôi hiểu caveat bulk HQL update/delete.

□ 14. Tôi có SQL/DB evidence cho persist, dirty checking, cache, detached và merge.

□ 15. Tôi biết course native Hibernate khác production framework-managed persistence ở đâu.

IX.1. Exit ticket

Câu hỏi	Trả lời
Vì sao SessionFactory không tạo mỗi CRUD operation?	
Transient → Persistent xảy ra khi nào?	
Flush có đồng nghĩa commit không?	
HQL `from STUDENTS` sai vì sao nếu entity là Student?	
Một evidence bạn dùng để chứng minh first-level cache?	

TÀI LIỆU THAM CHIẾU & SOURCE TRACEABILITY

- Course source: Chapter 17 – Hibernate Architecture, Configuration, Mapping. Nội dung: ORM, architecture, lifecycle, JPA vs Hibernate, configuration properties, annotations, relationships, HQL/Query API.
- Course source: Lab 07 – Building Desktop Application Using JavaFX and Hibernate. Baseline: Maven + Hibernate + SQL Server LibraryDB + Student/Book + Repository architecture.
- Official Hibernate ORM 7.4 documentation/User Guide – architecture, SessionFactory, Session, persistence context.
- Official Hibernate HQL Guide – current HQL language reference.
- Official Hibernate ORM 7.4 release page – compatibility with Java/Jakarta Persistence/Spring Boot.

NOTE: Current ecosystem note (verified October 2026): Hibernate ORM 7.4 is the latest stable series; the official release page lists 7.4.13.Final dated 2026-10-11, with Jakarta Persistence 3.2 and Spring Boot 4.1 compatibility. The course remains version-locked to its existing lab/project baseline unless the lecturer explicitly migrates it.